Lab. 1

(시계열 데이터) 이상치 탐지 실습

학습 내용

- **데이터 소개**
- **이상치 탐지 모델 실습**
  - 이상치 탐지 모델 학습
  - 탐지 결과 시각화
  - 성능 분석

데이터 소개

- **Synthetic 2D**
  - 목적
    - 이상치 탐지 개념 직관화, 모델별 경계 차이 확인 용도
  - 데이터 구성
    - 정상 데이터: 2개 가우시안 클러스터(서로 다른 중심/분산) 구성
    - 이상치 데이터: (1) 넓은 범위 균일 분포 이상치 + (2) 클러스터 사이 "애매 영역" 이상치 혼합
  - 라벨 정의
    - 정상: 0
    - 이상: 1
  - 학습/테스트 시나리오
    - 학습: 정상만 사용
    - 테스트: 정상 + 이상 혼합

데이터 소개

- **TwoMoons**
  - 목적
    - 비선형 정상 분포에서의 이상치 탐지 난이도 체감 및 모델 비교 용도
  - 데이터 구성
    - 정상 데이터: TwoMoons(두 개의 반달 모양) + 노이즈 포함
    - 이상치 데이터: 정상 분포를 둘러싼 박스 영역에서 균일 분포로 생성
  - 라벨 정의
    - 정상: 0
    - 이상: 1
  - 학습/테스트 시나리오
    - 학습: 정상만 사용
    - 테스트: 정상 + 이상 혼합

데이터 소개

- **AI4I 2020**
  - 목적
    - 표 데이터 기반 이상치 탐지 실습 및 성능 지표 비교 용도
  - 데이터 구성
    - 수치형 특징: 공정/센서 기반 수치 변수들 중심
    - 범주형 특징: 제품 타입(Type) 등 포함 가능
    - 타깃: Machine failure(고장) 컬럼 기반
  - 라벨 정의
    - 정상: Machine failure = 0
    - 이상: Machine failure = 1
  - 학습/테스트 시나리오
    - 학습: 정상만 사용
    - 테스트: 정상 + 이상 혼합

이상치 탐지 모델 실습

- 필요한 라이브러리 임포트

```
# ==========================================
# 실습 데이터: Synthetic2D / TwoMoons / AI4I
# ==========================================
!pip -q install scikit-learn matplotlib pandas numpy

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split

from sklearn.neighbors import LocalOutlierFactor
from sklearn.svm import OneClassSVM
from sklearn.ensemble import IsolationForest

from sklearn.preprocessing import StandardScaler, OrdinalEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline, make_pipeline

from sklearn.metrics import (
    precision_score, recall_score, f1_score,
    roc_auc_score, average_precision_score
)

from sklearn.decomposition import PCA
```

이상치 탐지 모델 실습

- **Fit scores 함수(모델 점수 계산)**

```python
# ----------------------------
# 0) Helpers: scoring/eval
# ----------------------------
def fit_scores(model, X_train_normal, X_test):
    """
    Fit model on normal-only training data.
    Return anomaly scores on X_test where larger => more anomalous.
    """
    model.fit(X_train_normal)

    if hasattr(model, "decision_function"):
        dec = model.decision_function(X_test).ravel()  # larger => more normal
        scores = -dec                                  # larger => more anomalous
    elif hasattr(model, "score_samples"):
        ss = model.score_samples(X_test).ravel()       # larger => more normal
        scores = -ss                                   # larger => more anomalous
    else:
        # fallback: use predict only
        raw = model.predict(X_test)
        y_pred = (raw == -1).astype(int)
        scores = y_pred.astype(float)

    return scores
```

- 정상 데이터만 사용한 모델 학습
- decision_function 또는 score_samples 기반 이상 점수 통일
- 점수 함수 미지원 모델에 대한 예외 처리

이상치 탐지 모델 실습

- **Threshold by rate, predict by threshold함수(임계값 산출 및 예측 생성)**

```python
def threshold_by_rate(scores, anomaly_rate):
    """
    Choose threshold so that top 'anomaly_rate' fraction are labeled anomalies.
    """
    anomaly_rate = float(anomaly_rate)
    anomaly_rate = max(0.0001, min(0.9999, anomaly_rate))  # keep safe
    thr = np.quantile(scores, 1.0 - anomaly_rate)
    return thr

def predict_by_threshold(scores, thr):
    return (scores >= thr).astype(int)
```

  - 이상치 비율 기반 임계값 산출(상위 anomaly_rate 비율에서 커팅)
  - 점수 임계값 기준 이상치 예측 벡터 생성
  - contamination 기반 공정 비교용 예측 규칙 통일

이상치 탐지 모델 실습

- **Evaluate 함수 (평가 지표 계산)**

```
def evaluate(y_true, y_pred, scores):
    return {
        "precision": precision_score(y_true, y_pred, zero_division=0),
        "recall":    recall_score(y_true, y_pred, zero_division=0),
        "f1":        f1_score(y_true, y_pred, zero_division=0),
        "roc_auc":   roc_auc_score(y_true, scores),
        "pr_auc":    average_precision_score(y_true, scores),
    }
```

  - 정답 라벨 기반 정밀도/재현율/F1 계산
  - 이상 점수 기반 ROC-AUC / PR-AUC 계산
  - 희소 이상치 환경에서 PR-AUC 중심 비교 지원

이상치 탐지 모델 실습

- **Plot points style 함수 (시각화 함수)**

```python
# ----------------------------
# 1) Visualization style (fixed)
# ----------------------------
def plot_points_style(X_train_normal_2d, X_test_2d, y_true_test, y_pred_test, title):
    """
    Style:
    '.' train normals
    'o' test normals
    'x' true anomalies
    ring (empty circle) predicted anomalies
    """
    plt.figure(figsize=(6.2, 5.2))

    plt.scatter(X_train_normal_2d[:, 0], X_train_normal_2d[:, 1],
                marker=".", alpha=0.55, label="Train normals")

    plt.scatter(X_test_2d[y_true_test == 0, 0], X_test_2d[y_true_test == 0, 1],
                marker="o", alpha=0.65, label="Test normals")

    plt.scatter(X_test_2d[y_true_test == 1, 0], X_test_2d[y_true_test == 1, 1],
                marker="x", alpha=0.9, label="True anomalies")

    out_idx = np.where(y_pred_test == 1)[0]
    plt.scatter(X_test_2d[out_idx, 0], X_test_2d[out_idx, 1],
                facecolors="none", edgecolors="k", s=90, label="Pred anomalies")

    plt.title(title)
    plt.xlabel("x1")
    plt.ylabel("x2")
    plt.grid(True, alpha=0.3)
    plt.legend()
    plt.show()
```

- Train 정상/ Test 정상/ True 이상/ Pred 이상 시각적 구분 수행
- 동일 마커 규칙 기반 데이터셋 간 비교 용이성 확보
- Pred 이상치 강조(속 빈 원) 처리

이상치 탐지 모델 실습

- **Plot boundary style 함수(1/2) (시각화 함수)**

```python
def plot_boundary_style(model, X_train_normal, X_test, y_true_test, y_pred_test, thr, title, grid_size=350):
    """
    2D only: draw boundary as contour of anomaly_score == threshold.
    anomaly_score is unified to be larger => more anomalous.
    """
    x_min, x_max = X_test[:, 0].min() - 0.8, X_test[:, 0].max() + 0.8
    y_min, y_max = X_test[:, 1].min() - 0.8, X_test[:, 1].max() + 0.8

    xx, yy = np.meshgrid(
        np.linspace(x_min, x_max, grid_size),
        np.linspace(y_min, y_max, grid_size)
    )
    grid = np.c_[xx.ravel(), yy.ravel()]

    # Grid scores (larger => more anomalous)
    if hasattr(model, "decision_function"):
        dec = model.decision_function(grid).ravel()
        grid_scores = (-dec).reshape(xx.shape)
    elif hasattr(model, "score_samples"):
        ss = model.score_samples(grid).ravel()
        grid_scores = (-ss).reshape(xx.shape)
    else:
        # no continuous score -> just points
        return plot_points_style(X_train_normal, X_test, y_true_test, y_pred_test, title)

    plt.figure(figsize=(6.2, 5.2))
```

- 2D 공간 grid 생성 및 모델 점수 계산
- 점수 임계값(thr)과 동일한 점수의 등고선 표시
- "상위 contamination 비율을 이상치로 간주" 기준에 따른 경계 시각화
- 연속 점수 미지원 모델에 대한 폴백 처리

이상치 탐지 모델 실습

- **Plot boundary style 함수(2/2) (시각화 함수)**

```
def plot_boundary_style(model, X_train_normal, X_test, y_true_test, y_pred_test, thr, title, grid_size=350):
            ⋮
    # boundary
    plt.contour(xx, yy, grid_scores, levels=[thr])

    # points with fixed style
    plt.scatter(X_train_normal[:, 0], X_train_normal[:, 1],
                marker=".", alpha=0.55, label="Train normals")

    plt.scatter(X_test[y_true_test == 0, 0], X_test[y_true_test == 0, 1],
                marker="o", alpha=0.65, label="Test normals")

    plt.scatter(X_test[y_true_test == 1, 0], X_test[y_true_test == 1, 1],
                marker="x", alpha=0.9, label="True anomalies")

    out_idx = np.where(y_pred_test == 1)[0]
    plt.scatter(X_test[out_idx, 0], X_test[out_idx, 1],
                facecolors="none", edgecolors="k", s=90, label="Pred anomalies")

    plt.title(title)
    plt.xlabel("x1")
    plt.ylabel("x2")
    plt.grid(True, alpha=0.3)
    plt.legend()
    plt.show()
```

- 2D 공간 grid 생성 및 모델 점수 계산
- 점수 임계값(thr)과 동일한 점수의 등고선 표시
- "상위 contamination 비율을 이상치로 간주" 기준에 따른 경계 시각화
- 연속 점수 미지원 모델에 대한 폴백 처리

이상치 탐지 모델 실습

- **Make models 2d 함수 (2D 데이터용 모델 구성)**

```python
# ----------------------------
# 2) Models
# ----------------------------
def make_models_2d(contamination=0.05, random_state=42):
    """
    2D는 스케일링이 유효해서 Pipeline(StandardScaler + model)로 구성.
    """
    return {
        "LOF": make_pipeline(
            StandardScaler(),
            LocalOutlierFactor(n_neighbors=20, novelty=True, contamination=contamination)
        ),
        "OneClassSVM": make_pipeline(
            StandardScaler(),
            OneClassSVM(kernel="rbf", nu=contamination, gamma="scale")
        ),
        "IsolationForest": make_pipeline(
            StandardScaler(),
            IsolationForest(n_estimators=300, contamination=contamination,
                            random_state=random_state, n_jobs=-1)
        ),
    }
```

- 거리/경계 민감 모델 대상 스케일링 적용 파이프라인 구성
- LOF novelty 모드 기반 신규 샘플 점수화 가능 설정
- contamination/nu 기반 이상치 비율 반영 설정

이상치 탐지 모델 실습

- **Make models tabular 함수 (테이블 데이터용 모델 구성)**

```python
def make_models_tabular(contamination=0.05, random_state=42):
    """
    AI4I는 이미 전처리에서 수치 스케일링을 하므로(그리고 범주형도 포함되므로)
    모델 앞에 추가 StandardScaler는 생략(교육용 단순/일관성).
    """
    return {
        "LOF": LocalOutlierFactor(n_neighbors=20, novelty=True, contamination=contamination),
        "OneClassSVM": OneClassSVM(kernel="rbf", nu=contamination, gamma="scale"),
        "IsolationForest": IsolationForest(n_estimators=300, contamination=contamination,
                                           random_state=random_state, n_jobs=-1),
    }
```

  - AI4I 전처리 단계에서 스케일링 수행 전제의 모델 구성
  - 모델 자체 입력을 전처리 결과 행렬로 직접 사용

이상치 탐지 모델 실습

- **Make synthetic 2d 함수 (Synthetic 2D 생성)**

```python
# ----------------------------
# 3) Dataset A: Synthetic 2D
# ----------------------------
def make_synthetic_2d(seed=42, n_norm1=600, n_norm2=400, n_out=120):
    """
    Normal: two Gaussian blobs
    Outliers: uniform in a wide box + small "gap" anomalies (optional effect)
    """
    rng = np.random.default_rng(seed)

    # normals (two clusters)
    norm1 = rng.normal(loc=(-2.0, -2.0), scale=(0.6, 0.6), size=(n_norm1, 2))
    norm2 = rng.normal(loc=(2.5, 2.0),  scale=(0.35, 0.35), size=(n_norm2, 2))
    X_norm = np.vstack([norm1, norm2])

    # anomalies
    out_far = rng.uniform(low=-7, high=7, size=(int(n_out * 0.7), 2))
    out_gap = rng.normal(loc=(0.5, 0.2), scale=(0.6, 0.6), size=(n_out - len(out_far), 2))
    X_out = np.vstack([out_far, out_gap])

    X = np.vstack([X_norm, X_out])
    y = np.hstack([
        np.zeros(len(X_norm), dtype=int),
        np.ones(len(X_out), dtype=int)
    ])
    return X, y
```

- 2개 정상 군집 데이터 생성(가우시안)
- 전역 이상치(균일 분포) + 애매 영역 이상치 (정규 분포) 혼합 생성
- 정상/이상 라벨 벡터 생성

이상치 탐지 모델 실습

- **Synthetic 2D 실험 실행(학습/평가/경계 시각화)**

```python
print("====================================")
print("A) Synthetic 2D")
print("====================================")

X, y = make_synthetic_2d(seed=42)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.35, random_state=42, stratify=y
)

X_train_normal = X_train[y_train == 0]
contamination = float(y_train.mean())

models = make_models_2d(contamination=contamination, random_state=42)

rows = []
for name, model in models.items():
    scores = fit_scores(model, X_train_normal, X_test)
    thr = threshold_by_rate(scores, contamination)
    y_pred = predict_by_threshold(scores, thr)

    rows.append({"dataset": "Synthetic2D", "model": name, **evaluate(y_test, y_pred, scores)})

    # boundary + style plot
    plot_boundary_style(
        model=model,
        X_train_normal=X_train_normal,
        X_test=X_test,
        y_true_test=y_test,
        y_pred_test=y_pred,
        thr=thr,
        title=f"[Synthetic2D] {name} (boundary = score threshold)"
    )

synthetic_results = pd.DataFrame(rows).sort_values(["pr_auc", "f1"], ascending=False)
display(synthetic_results)
```

- 데이터 분할 및 정상 데이터만 추출
- 라벨 비율 기반 contamination 산출
- 모델별 점수 계산 → 임계값 산출 → 예측 생성
- 성능 지표 테이블 생성 및 정렬
- 2D boundary(등고선) + 고정 스타일 점 표시 시각화

이상치 탐지 모델 실습

- **TwoMoons 데이터 생성(정상) + 이상치 주입**

```
# ---------------------------
# 4) Dataset B: TwoMoons
# ---------------------------
print("====================================")
print("B) TwoMoons")
print("====================================")

rng = np.random.default_rng(42)

X_normal, _ = make_moons(n_samples=800, noise=0.08, random_state=42)
n_out = 80
X_out = rng.uniform(low=-1.5, high=2.5, size=(n_out, 2))

X = np.vstack([X_normal, X_out])
y = np.hstack([np.zeros(len(X_normal), dtype=int), np.ones(len(X_out), dtype=int)])

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.35, random_state=42, stratify=y
)

X_train_normal = X_train[y_train == 0]
contamination = float(y_train.mean())
```

- 비선형 정상 분포(TwoMoons) 생성
- 정상 분포 주변 박스 영역에 균일 이상치 주입
- 정상/이상 라벨 생성
- 정상만 학습(train 정상) 구성

이상치 탐지 모델 실습

- **TwoMoons 실험 실행(학습/평가/경계 시각화)**

```
models = make_models_2d(contamination=contamination, random_state=42)

rows = []
for name, model in models.items():
    scores = fit_scores(model, X_train_normal, X_test)
    thr = threshold_by_rate(scores, contamination)
    y_pred = predict_by_threshold(scores, thr)

    rows.append({"dataset": "TwoMoons", "model": name, **evaluate(y_test, y_pred, scores)})

    plot_boundary_style(
        model=model,
        X_train_normal=X_train_normal,
        X_test=X_test,
        y_true_test=y_test,
        y_pred_test=y_pred,
        thr=thr,
        title=f"[TwoMoons] {name} (boundary = score threshold)"
    )

twomoons_results = pd.DataFrame(rows).sort_values(["pr_auc", "f1"], ascending=False)
display(twomoons_results)
```

- 모델별 이상 점수 → 임계값 → 예측 생성
- 비선형 구조에서 모델별 경계 형태 차이 시각화
- PR-AUC/F1 중심 성능 비교 테이블 산출

이상치 탐지 모델 실습

- **AI4I 데이터 업로드 및 컬럼 구성(타깃/누수 제거)**

```
# ----------------------------
# 5) Dataset C: AI4I (Upload CSV)
# ----------------------------
print("=====================================")
print("C) AI4I2020 (Upload ai4i2020.csv)")
print("=====================================")

from google.colab import files
uploaded = files.upload()
ai4i_path = next(iter(uploaded.keys()))

df = pd.read_csv(ai4i_path)

TARGET = "Machine failure"
LEAK = ["TWF", "HDF", "PWF", "OSF", "RNF"]  # leakage columns (failure types)
DROP = ["UDI", "Product ID"] + LEAK

feature_cols = [c for c in df.columns if c not in DROP + [TARGET]]
X_df = df[feature_cols].copy()
y = df[TARGET].astype(int).values  # 1 = anomaly

cat_cols = [c for c in feature_cols if X_df[c].dtype == "object"]
num_cols = [c for c in feature_cols if c not in cat_cols]

print("AI4I features:", feature_cols)
print("cat_cols:", cat_cols, "| num_cols:", num_cols)
print("anomaly ratio (Machine failure=1):", y.mean())
```

- Colab 파일 업로드 및 CSV 로딩
- 타깃 컬럼 및 누수 컬럼 제거 목록 구성
- 학습 입력 feature 행렬 및 이상 라벨 벡터 생성
- 범주형/수치형 feature 자동 분리

이상치 탐지 모델 실습

- **AI4I PCA 2D 투영(시각화를 위한 차원 축소)**

```
models = make_models_tabular(contamination=contamination, random_state=42)

# PCA for visualization only (fit on train normals)
pca = PCA(n_components=2, random_state=42)
X_train_normal_2d = pca.fit_transform(X_train_normal)
X_test_2d = pca.transform(X_test_all)
```

  - 고차원 데이터 2차원 투영을 통한 시각화 보조
  - PCA fit을 train 정상에만 적용하여 투영 기준 안정화
  - 원공간 경계 대신 투영 공간 분포 확인 목적

이상치 탐지 모델 실습

- **AI4I 실험 실행(학습/평가/투영 시각화)**

```
rows = []
for name, model in models.items():
    scores = fit_scores(model, X_train_normal, X_test_all)
    thr = threshold_by_rate(scores, contamination)
    y_pred = predict_by_threshold(scores, thr)

    rows.append({"dataset": "AI4I2020", "model": name, **evaluate(y_test, y_pred, scores)})

    # PCA 2D projection + fixed style (no boundary in PCA space)
    plot_points_style(
        X_train_normal_2d=X_train_normal_2d,
        X_test_2d=X_test_2d,
        y_true_test=y_test,
        y_pred_test=y_pred,
        title=f"[AI4I PCA2D] {name} (x=true failure, ring=pred anomaly)"
    )

ai4i_results = pd.DataFrame(rows).sort_values(["pr_auc", "f1"], ascending=False)
display(ai4i_results)
```

- 정상 값 학습 기반 모델 적합
- 점수 기반 임계값 예측 규칙 적용
- 고정 스타일 기반 PCA 2D 시각화
- 데이터셋 내 모델 성능 비교 테이블 산출

이상치 탐지 모델 실습

- **최종 결과 요약 테이블 생성(데이터셋별 성능 + 피처 수)**

```
# ----------------------------
# 6) Summary table (with n_features)
# ----------------------------
# feature counts for each dataset
n_features_map = {
    "Synthetic2D": X_train.shape[1],      # should be 2
    "TwoMoons": X_train.shape[1],         # should be 2
    "AI4I2020": X_train_all.shape[1],     # after preprocessing
}

all_results = pd.concat([synthetic_results, twomoons_results, ai4i_results], ignore_index=True)

# add feature count column
all_results["n_features"] = all_results["dataset"].map(n_features_map).astype(int)

print("=====================================")
print("Summary (sorted by dataset, then PR-AUC, F1)")
print("=====================================")
display(all_results.sort_values(["dataset", "pr_auc", "f1"], ascending=[True, False, False]))
```

- 데이터셋별 feature 차원 수 집계 및 매핑
- 데이터셋별 결과 테이블 병합
- feature 수 포함 통합 요약 테이블 생성
- 데이터셋별 PR-AUC/F1 기준 정렬 및 출력

HANYANG
UNIVERSITY